DYP DPU

Dr. D. Y. Patil Arts, Commerce & Science College, Pimpri

Department of Computer Science
2025-2026

T.Y.B.C.A. (Science) Sem-VI

Subject: -Programming in GO

Practical Assignment 4

1. Write a program in GO language to print file information.

Sol :-
```
package main
import (
    "fmt"
    "os"
    "time"
)
func main() {
    var filename string
    fmt.Print("Enter file name: ")
    fmt.Scanln(&filename)
    // Get file info
    info, err := os.Stat(filename)
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println("\n--- File Information ---")
    fmt.Println("Name       :", info.Name())
    fmt.Println("Size       :", info.Size(), "bytes")
    fmt.Println("Permission:", info.Mode())
    fmt.Println("Modified   :", info.ModTime().Format(time.RFC1123))
    fmt.Println("Is Directory:", info.IsDir())
}
```

2. Write a program in GO language program to create Text file.

Sol: - package main

```
import (
    "fmt"
    "os"
)
func main() {
    // Create text file
    file, err := os.Create("sample.txt")
    if err != nil {
        fmt.Println("Error creating file:", err)
        return
    }
    // Always close file
    defer file.Close()

    // Write text into file
    content := "Hello Students!\nThis file is created using Go language."
    _, err = file.WriteString(content)
    if err != nil {
        fmt.Println("Error writing to file:", err)
        return
    }
    fmt.Println("Text file created successfully!")
}
```

3. Write a function in GO language to find the square of a number and write a benchmark for it.

Sol:- 1. Create Square.go

```
Package  square

func Square(n int) int {

        return n * n

}
```

2. square_test.go

```
package square

import "testing"
```

```
func BenchmarkSquare(b *testing.B) {
    for i := 0; i < b.N; i++ {
        Square(10)
    }
}
```

4. Write a program in GO language to read XML file into structure and display structure.

Sol:- student.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<Student>

  <ID>101</ID>

  <Name>Bharati Bhamarel</Name>

  <Course>BCA</Course>

</Student>
```

Main.go

```
package main

import (
        "encoding/xml"
        "fmt"
        "io/ioutil"
        "os"
)

// Define structure matching XML
type Student struct {
        XMLName xml.Name `xml:"Student"`
        ID      int     `xml:"ID"`
        Name    string  `xml:"Name"`
        Course  string  `xml:"Course"`
}

func main() {

        // Open XML file
        file, err := os.Open("student.xml")
        if err != nil {
```

```
		fmt.Println("Error opening file:", err)
		return
	}
	defer file.Close()

	// Read file content
	data, err := ioutil.ReadAll(file)
	if err != nil {
		fmt.Println("Error reading file:", err)
		return
	}

	// Create structure variable
	var s Student

	// Unmarshal XML into structure
	err = xml.Unmarshal(data, &s)
	if err != nil {
		fmt.Println("Error unmarshalling XML:", err)
		return
	}

	// Display structure
	fmt.Println("Student Details:")
	fmt.Println("ID     :", s.ID)
	fmt.Println("Name   :", s.Name)
	fmt.Println("Course :", s.Course)
}
```

5. Write a program in GO language to create a user defined package to find out the area of a rectangle.

Sol:- rectangle.go

```
package rectangle
// Area calculates area of rectangle
func Area(length, width float64) float64 {
   return length * width
}
```

Main.go

```
package main
import (
   "fmt"
   "rectarea/rectangle"
```

```
)
func main() {
    var l, w float64
    fmt.Print("Enter length: ")
    fmt.Scanln(&l)

    fmt.Print("Enter width: ")
    fmt.Scanln(&w)
    area := rectangle.Area(l, w)
    fmt.Println("Area of Rectangle =", area)
}
```

6. Write a program in GO language to add or append content at the end of a text file.

Sol:- package main

```
import (
        "fmt"
        "os"
)
func main() {

        // File name
        filename := "sample.txt"

        // Text to append
        text := "\nThis line is appended using Go program."

        // Open file in append mode
        file, err := os.OpenFile(filename, os.O_APPEND|os.O_WRONLY, 0644)
        if err != nil {
                fmt.Println("Error opening file:", err)
                return
        }
        defer file.Close()

        // Write text to file
        _, err = file.WriteString(text)
        if err != nil {
                fmt.Println("Error writing to file:", err)
                return
        }

        fmt.Println("Content appended successfully!")
}
```

7. Write a program in GO language using a user defined package calculator that

performs one calculator operation as per the user's choice.

Sol:- cal.go

```
package calculator

// Addition
func Add(a, b int) int {
   return a + b
}

// Subtraction
func Sub(a, b int) int {
   return a - b
}

// Multiplication
func Mul(a, b int) int {
   return a * b
}

// Division
func Div(a, b int) int {
   return a / b
}
```

Main.go

```
  package main

import (
		"fmt"
		"calculatorApp/calculator"
)

func main() {

		var a, b, choice int

		fmt.Println("Enter two numbers:")
		fmt.Scan(&a, &b)

		fmt.Println("\n1.Addition\n2.Subtraction\n3.Multiplication\n4.Division")
		fmt.Print("Enter your choice: ")
		fmt.Scan(&choice)

		switch choice {
		case 1:
			fmt.Println("Result =", calculator.Add(a, b))
```

```
		case 2:
			fmt.Println("Result =", calculator.Sub(a, b))

		case 3:
			fmt.Println("Result =", calculator.Mul(a, b))

		case 4:
			fmt.Println("Result =", calculator.Div(a, b))

		default:
			fmt.Println("Invalid Choice")
		}
	}
```

8. Write a program in the GO language program to open a file in READ only mode.

Sol:- package main

```
import (
		"fmt"
		"os"
)

func main() {

		// Open file in READ ONLY mode
		file, err := os.Open("data.txt") // os.Open opens file as read-only
		if err != nil {
			fmt.Println("Error opening file:", err)
			return
		}
		defer file.Close()

		fmt.Println("File opened successfully in READ ONLY mode.")

		// Read file content
		buffer := make([]byte, 100)

		n, err := file.Read(buffer)
		if err != nil {
			fmt.Println("Error reading file:", err)
			return
		}

		fmt.Println("File Content:")
```

```
            fmt.Println(string(buffer[:n]))
}
```

9. WAP to create student struct with student name and marks and sort it based on student marks using sort package.

Sol:-

```
package main

import (
   "fmt"
   "sort"
)

// Define Student structure
type Student struct {
   Name  string
   Marks int
}

func main() {

   // Create slice of students
   students := []Student{
      {"Rahul", 75},
      {"Anita", 90},
      {"Suresh", 68},
      {"Meena", 82},
   }

   fmt.Println("Before Sorting:")
   for _, s := range students {
      fmt.Println(s.Name, "-", s.Marks)
   }

   // Sort students based on Marks
   sort.Slice(students, func(i, j int) bool {
      return students[i].Marks < students[j].Marks
   })

   fmt.Println("\nAfter Sorting (By Marks):")
   for _, s := range students {
      fmt.Println(s.Name, "-", s.Marks)
   }
}
```

10. WAP in Go language to add two integers and write code for unit test to test this code.

Sol:- main.go

```
package main
```

```
import "fmt"

// Add function
func Add(a int, b int) int {
   return a + b
}

func main() {
   x := 10
   y := 20

   result := Add(x, y)
   fmt.Println("Sum =", result)
}
```

Main_test.go

```
package main
import "testing"
// Unit test for Add()
func TestAdd(t *testing.T) {

   result := Add(10, 20)
   expected := 30

   if result != expected {
     t.Errorf("Add(10,20) = %d; want %d", result, expected)
   }
}
```

11. WAP in Go language to subtract two integers and write code for table test to test this code.

Sol:- main.go

```
package main

import "fmt"

// Subtract function
func Subtract(a, b int) int {
   return a - b
}

func main() {
   x := 20
   y := 10

   result := Subtract(x, y)
```

```
    fmt.Println("Subtraction =", result)
}
```

Main_test.go

```
  package main

import "fmt"

// Subtract function
func Subtract(a, b int) int {
    return a - b
}

func main() {
    x := 20
    y := 10

    result := Subtract(x, y)
    fmt.Println("Subtraction =", result)
}
```

12. WAP in Go language to create an user defined package to find out the area of a Square.

Sol:- sq.go

```
     package sq

// Area function to calculate area of square
func Area(side float64) float64 {
      return side * side
}
```

Main.go package main

```
import (
      "fmt"
      "square/sq"
)

func main() {

      var side float64

      fmt.Print("Enter side of square: ")
```

```
    fmt.Scanln(&side)

    area := sq.Area(side)

    fmt.Println("Area of Square =", area)
}
```